Build Your Own

10-Week Hardcore CS Fundamentals Plan

C++ · OS · Networking · IR · DBMS · NLP · Docker

Enhanced Edition v2 — Expanded with blueprint alignment: additional tutorials, implementation tasks, system design deep-dives, Indian language NLP, evaluation frameworks, and deployment strategies.

Strategy: Learn a concept → implement it the same day. Never finish a full course before building. Systems depth comes from doing.

■ Concepts	■ Resources	⌖ Build Tasks	■ Critical Note	★ New Addition

Table of Contents

PHASE 0 (Day 0) — Setup + Mental Model

Day 0 — Environment + Blueprint

Orient yourself before writing a single line of code. Install your full toolchain, read the entire project blueprint, and sketch the data-flow diagram on paper.

Core Concepts:

- Install: gcc/g++, cmake, docker, postgres, redis, valgrind, wireshark/tcpdump
- Verify: g++ --version, docker ps, psql --version
- Sketch: crawler → raw HTML → NLP pipeline → inverted index → query engine → API → UI
- ★ Full data-flow: Seed URLs → Redis URL Frontier → C++ Crawler Workers → Raw HTML Store (MinIO) → Kafka → Python NLP Pipeline → Inverted Index Builder → Index Shards (PG + RocksDB) → Query Engine → FastAPI → Nginx

■ Deep-Dive Resources:

■ learncpp.com — start here — https://www.learncpp.com/
■ The Missing Semester of CS (MIT) — https://missing.csail.mit.edu/
■ Project Blueprint PDF — distributed_search_engine_blueprint.pdf
■ GitHub: pclub.in roadmaps — https://pclub.in/roadmaps
■ ★ CommonCrawl — how production crawlers work — https://commoncrawl.org/the-data/how-it-works/
■ ★ Google Search Architecture (Jeff Dean talk) — https://research.google/pubs/pub38331/

✥ Build Tasks:

- ✥ Read blueprint fully end-to-end
- ✥ Install all tools and verify
- ✥ Sketch data-flow on paper/whiteboard
- ✥ Create GitHub repo with folder structure from blueprint
- ✥ ★ Write a one-paragraph description of each microservice (crawler, pipeline, indexer, query, frontend) in your own words
- ✥ ★ Identify the CAP theorem position of each store (Redis = CP for frontier, Postgres + RocksDB = AP for index)

■ **Do NOT skip this day. Architecture clarity now saves days of refactoring later.**

WEEK 1 — C++ Core: Pointers, Memory, STL

Day 1 — Pointers & References

Pointers are the foundation of everything in systems — memory management, data structures, OS interactions, and your crawler. Master pointer arithmetic before moving on.

Core Concepts:

- Raw pointers vs references — when to use which
- Pointer arithmetic — traversing arrays, stride-based access
- Pass-by-pointer vs pass-by-reference — semantics and performance
- NULL vs nullptr — prefer nullptr in modern C++
- void* and casting — when the type system gets in the way
- ★ Pointer-to-member and function pointers — used in thread pool callbacks

■ Deep-Dive Resources:

■ learncpp.com — Pointers & References — https://www.learncpp.com/cpp-tutorial/introduction-to-pointers/
■ GeeksforGeeks — C++ Pointers (comprehensive) — https://www.geeksforgeeks.org/cpp-pointers/
■ The Cherno — Pointers in C++ (YouTube) — https://www.youtube.com/watch?v=DTxHyVn0ODg
■ cppreference — Pointer declaration — https://en.cppreference.com/w/cpp/language/pointer
■ CS50 — Memory (YouTube, Harvard) — https://www.youtube.com/watch?v=NKTfNv2T0FE
■ ★ cppreference — Function pointers — https://en.cppreference.com/w/cpp/language/pointer#Pointers_to_functions

✦ Build Tasks:

- ✦ Reverse an array using pointer arithmetic only (no index variable)
- ✦ Build a dynamic array struct using new/delete
- ✦ Write swap() using pointers and swap() using references — compare assembly output
- ✦ ★ Implement a simple callback system using function pointers (simulate task queue)

■ Pointers are why C++ is used for systems. You MUST be fluent before Day 2.

Day 2 — Stack vs Heap + Memory Management

Understand where your data lives. Stack is fast but limited; heap is flexible but manually managed. Mismanage it and your crawler will silently corrupt memory.

Core Concepts:

- Stack frame lifecycle — push/pop, frame pointer, return address
- Heap allocation with new/delete and new[]/delete[]
- Memory leaks — how they happen, how to detect them
- Buffer overflow basics — why they corrupt memory silently
- Fragmentation — internal vs external, slab allocators
- ★ AddressSanitizer (ASan) — compile-time instrumentation for memory errors

■ Deep-Dive Resources:

■ learncpp — Dynamic Memory Allocation — https://www.learncpp.com/cpp-tutorial/dynamic-memory-allocation-with-new-and-delete/
■ Stack vs Heap — StackOverflow canonical — https://stackoverflow.com/questions/79923/what-and-where-are-the-stack-and-heap

■ Valgrind Quickstart — https://valgrind.org/docs/manual/quick-start.html
■ The Cherno — Stack vs Heap Memory (YouTube) — https://www.youtube.com/watch?v=wJ1L2nSIV1s
■ MIT OCW 6.172 — Memory (lecture notes) — https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/
■ ★ Google AddressSanitizer documentation — https://github.com/google/sanitizers/wiki/AddressSanitizer

❖ Build Tasks:

- ❖ Simulate a memory leak intentionally (allocate, never free)
- ❖ Detect it with valgrind --leak-check=full
- ❖ Fix it with delete[] — observe valgrind output change
- ❖ Write a stack-based string class and observe stack overflow with deep recursion
- ❖ ★ Compile with -fsanitize=address and reproduce a buffer overflow — observe ASan report

Day 3 — RAII + Smart Pointers

Modern C++ manages memory automatically using RAII. Your crawler will use smart pointers exclusively — this is how production C++ is written since C++11.

Core Concepts:

- RAII principle — resource acquisition is initialization
- unique_ptr — exclusive ownership, zero overhead vs raw pointer
- shared_ptr — reference counting, control block overhead
- weak_ptr — break reference cycles (e.g. doubly-linked list nodes)
- Move semantics with std::move — transfer ownership without copy
- ★ make_unique / make_shared — prefer over raw new for exception safety

■ Deep-Dive Resources:

■ learncpp — std::unique_ptr — https://www.learncpp.com/cpp-tutorial/stdunique_ptr/
■ learncpp — std::shared_ptr — https://www.learncpp.com/cpp-tutorial/stdshared_ptr/
■ cppreference — Memory management — https://en.cppreference.com/w/cpp/memory
■ The Cherno — Smart Pointers in C++ (YouTube) — https://www.youtube.com/watch?v=UOB7-B2MfwA
■ Fluent C++ — Smart Pointers deep dive — https://www.fluentcpp.com/2017/08/22/smart-developers-use-smart-pointers-smart-pointers-basics/
■ ★ CppCon 2019 — Back to Basics: Smart Pointers (YouTube) — https://www.youtube.com/watch?v=xGDLkt-jBJ4

❖ Build Tasks:

- ❖ Rewrite Day 1 dynamic array using unique_ptr
- ❖ Build a shared resource graph with shared_ptr and show ref counts
- ❖ Demonstrate dangling pointer bug, then fix it with smart pointer
- ❖ ★ Implement a RAII file-handle wrapper (constructor opens, destructor closes) — used for raw HTML dump files in the crawler

■ **Your crawler will use unique_ptr everywhere. No raw owning pointers in production code.**

Day 4 — STL Deep Dive

STL containers are your daily tools. Knowing their internals — not just their API — determines whether your code is fast or slow.

Core Concepts:

- vector — contiguous memory, amortized O(1) push_back, reserve() to pre-allocate

- unordered_map — hash table, O(1) avg lookup, rehashing cost
- map — red-black tree, O(log n) ordered operations
- queue/deque — deque is segmented array, good for BFS frontiers
- Iterators and range-based for — how they work under the hood
- ★ std::priority_queue — used in URL frontier priority scheduling (BFS vs best-first)

■ Deep-Dive Resources:

■ cppreference — STL Containers — https://en.cppreference.com/w/cpp/container
■ GeeksforGeeks — STL Complete Reference — https://www.geeksforgeeks.org/the-c-standard-template-library-stl/
■ The Cherno — STL Playlist (YouTube) — https://www.youtube.com/playlist?list=PLlrATfBNZ98dudnM48yfGUldqGD0S4FFb
■ Hacking C++ — STL cheatsheet (visual) — https://hackingcpp.com/cpp/cheat_sheets.html
■ CppCon — std::unordered_map internals (YouTube) — https://www.youtube.com/watch?v=fHNmRkzxHWs
■ ★ cppreference — std::priority_queue — https://en.cppreference.com/w/cpp/container/priority_queue

✦ Build Tasks:

- ✦ Implement word frequency counter on 1MB text with unordered_map
- ✦ Build a priority queue (max-heap) for URL scheduling by priority score
- ✦ Benchmark vector.push_back vs reserve+push_back — measure ns/op
- ✦ ★ Simulate the URL frontier: priority_queue> where float is estimated PageRank; verify top-5 URLs are highest-scored

Day 5 — Time Complexity + Cache Locality

Why arrays beat linked lists in real systems — cache lines, prefetching, and memory layout matter enormously for search engine performance.

Core Concepts:

- Big-O analysis — average vs worst case, amortized complexity
- CPU cache hierarchy — L1 (~4 cycles), L2 (~12 cycles), L3 (~40 cycles), RAM (~200 cycles)
- Cache line size — 64 bytes, spatial locality principle
- False sharing in multithreading — two threads on same cache line cause contention
- Data-Oriented Design — SoA vs AoS layout
- ★ Posting list iteration is cache-critical — delta-encoded sorted arrays beat hashmaps for sequential access

■ Deep-Dive Resources:

■ Scott Meyers — CPU Caches and Why You Care (CppCon) — https://www.youtube.com/watch?v=WDIkqP4JbkE
■ Gallery of Processor Cache Effects — Igor Ostrovsky — https://igoro.com/archive/gallery-of-processor-cache-effects/
■ MIT OCW 6.172 — Lecture 14: Caching (PDF slides) — https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/pages/lecture-slides/
■ Chandler Carruth — Efficiency with Algorithms (CppCon) — https://www.youtube.com/watch?v=fHNmRkzxHWs
■ ★ What Every Programmer Should Know About Memory — Ulrich Drepper (PDF) — https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

✦ Build Tasks:

- ✦ Benchmark vector vs list iteration — measure ns/element with std::chrono
- ✦ Compare struct-of-arrays vs array-of-structs on 1M elements
- ✦ Observe false sharing: two threads increment adjacent variables vs separate cache lines
- ✦ ★ Benchmark sequential scan of a sorted int array (delta-encoded posting list simulation) vs random access in unordered_map — quantify the cache miss penalty

Day 6 — File I/O + Serialisation

Your crawler saves HTML to disk. Understanding buffered vs unbuffered I/O and binary file formats is essential for building efficient crawl storage.

Core Concepts:

- fstream: ifstream, ofstream, fstream — open modes, flush, close
- Buffered I/O — setvbuf, std::ios::sync_with_stdio
- Binary vs text mode — endianness, struct packing, padding
- Memory-mapped files (mmap) concept — zero-copy file access
- Endianness in binary files — htonl/ntohl for portable formats
- ★ Content-addressed storage: SHA256(body) as filename — same page crawled twice → same hash → no duplicate storage

■ Deep-Dive Resources:

■ learncpp — File I/O — https://www.learncpp.com/cpp-tutorial/basic-file-io/
■ cppreference — fstream — https://en.cppreference.com/w/cpp/io/basic_fstream
■ Linux man page — mmap(2) — https://man7.org/linux/man-pages/man2/mmap.2.html
■ Johnny's Software Lab — mmap tutorial — https://johnnysswlab.com/memory-mapped-files/
■ Julia Evans — Profiling with perf (file I/O section) — https://jvns.ca/perf-zine.pdf
■ ★ MinIO Python SDK — object storage for raw HTML blobs — https://min.io/docs/minio/linux/developers/python/API.html

✦ Build Tasks:

- ✦ Write crawled HTML pages to disk with content-addressed filenames (SHA256 hash)
- ✦ Read them back, parse first 100 bytes, measure throughput in MB/s
- ✦ Implement mmap-based file reader and compare speed vs fstream
- ✦ ★ Simulate MinIO-style content-addressed store: dict[sha256] → filepath; verify inserting same HTML twice stores it only once

Day 7 — Mini Project: LRU Cache

The LRU Cache combines unordered_map + doubly-linked list for O(1) get/put. This exact pattern appears in your URL cache and DNS cache.

Core Concepts:

- LRU eviction policy — Least Recently Used
- HashMap + DLL combo — map stores iterators into the list
- O(1) get and O(1) put — amortized with good hash function
- Cache hit/miss ratio — measure and tune
- Thread-safe LRU — add mutex for crawler use
- ★ TTL-aware LRU — DNS entries expire; add expiry timestamps as in blueprint DNSEntry struct

■ Deep-Dive Resources:

■ LeetCode 146 — LRU Cache (canonical problem) — https://leetcode.com/problems/lru-cache/
■ Nick White — LRU Cache solution walkthrough (YouTube) — https://www.youtube.com/watch?v=NDpwj0VWz1U
■ GeeksforGeeks — LRU Cache implementation — https://www.geeksforgeeks.org/lru-cache-implementation/
■ cppreference — std::list (for the DLL) — https://en.cppreference.com/w/cpp/container/list
■ ★ Blueprint DNSEntry struct — TTL-aware LRU pattern — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Implement LRU Cache from scratch — no hints, no library
- ✦ Test with 1M random get/put operations, measure throughput
- ✦ Add thread-safety with std::mutex, re-benchmark

- ✥ ★ Extend to TTL-aware LRU: entries expire after N seconds (replicate blueprint DNSResolver::resolve() pattern)
- ✥ ★ Wire this cache into a stub DNS resolver — resolve a hostname, cache the IP for 300 s, observe cache hit on second call

■ **Week 1 Output: LRU Cache + memory-safe dynamic array with smart pointers.**

WEEK 2 — OS + Threading + Concurrency

Day 8 — Processes vs Threads

Your crawler is multi-threaded. Understand what the OS gives each thread and what's shared — this determines your entire concurrency design.

Core Concepts:

- Process: private address space, PCB, file descriptor table
- Thread: shared heap/globals/fds, private stack/registers/TLS
- Context switching overhead — register save/restore, TLB flush
- User threads vs kernel threads — 1:1 vs M:N mapping
- fork() vs thread creation cost — fork copies page tables (CoW)
- ★ Thread-local storage (TLS) — per-thread HTTP fetcher and DNS resolver instances avoid locking

■ Deep-Dive Resources:

■ OSTEP — Processes chapter (free PDF) — https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf
■ OSTEP — Threads Intro (free PDF) — https://pages.cs.wisc.edu/~remzi/OSTEP/threads-intro.pdf
■ GeeksforGeeks — Process vs Thread — https://www.geeksforgeeks.org/difference-between-process-and-thread/
■ CS162 Berkeley — Lecture 4: Processes & Threads (YouTube) — https://www.youtube.com/watch?v=XSaj4PtMXeg
■ Linux man page — fork(2) — https://man7.org/linux/man-pages/man2/fork.2.html
■ ★ cppreference — thread_local storage — https://en.cppreference.com/w/cpp/keyword/thread_local

✦ Build Tasks:

✦ Write a process that fork()s 10 times and measures fork() latency with clock_gettime()
✦ Compare with thread creation time — expect 10-100x difference
✦ Use /proc/{pid}/maps to visualize address space layout
✦ ★ Declare a thread_local DNSResolver instance; spawn 8 threads, each resolving a different hostname — confirm no locking needed and no data races under TSan

Day 9 — Threading in C++

std::thread is your primitive. Understand its lifecycle, joinable state, and detach — misuse leads to silent crashes (std::terminate on destructor of joinable thread).

Core Concepts:

- std::thread — creation, join(), detach(), lifecycle states
- Lambda captures in threads — capture by value vs reference (dangling refs!)
- std::this_thread::sleep_for, get_id(), hardware_concurrency()
- Thread lifecycle: new → runnable → running → blocked → terminated
- RAII thread wrapper — join-on-destruction pattern
- ★ std::jthread (C++20) — automatically joins on destruction, supports stop tokens

■ Deep-Dive Resources:

■ cppreference — std::thread — https://en.cppreference.com/w/cpp/thread/thread
■ learncpp — Introduction to threads — https://www.learncpp.com/cpp-tutorial/introduction-to-threads/
■ The Cherno — Threads in C++ (YouTube) — https://www.youtube.com/watch?v=wXBcwHwIt_I
■ Anthony Williams — C++ Concurrency in Action (book preview) — https://www.manning.com/books/c-plus-plus-concurrency-in-action

■ Educative.io — Multithreading in C++ (free article) — https://www.educative.io/blog/modern-multithreading-and-concurrency-in-cpp
■ ★ cppreference — std::jthread (C++20) — https://en.cppreference.com/w/cpp/thread/jthread

❖ Build Tasks:

- ❖ Spawn 10 threads, each fetching a URL segment in sequence
- ❖ Benchmark sequential vs parallel execution with std::chrono
- ❖ Deliberately leak a joinable thread — observe std::terminate
- ❖ ★ Re-implement using std::jthread with stop_token — observe clean cancellation when the token is requested

Day 10 — Mutex + Locks + Race Conditions

Shared data in your crawler (URL queue, visited set) needs protection. Race conditions are silent killers — they manifest under load, not in tests.

Core Concepts:

- Race condition: classic bank account / shared counter example
- std::mutex: lock() / unlock() — always pair them
- std::lock_guard — RAII scoped lock, exception-safe
- std::unique_lock — flexible: deferred, timed, conditional
- Deadlock: circular wait, mutual exclusion, hold-and-wait — prevention via lock ordering
- ★ std::shared_mutex — multiple readers / single writer for DNS cache (read-mostly)

■ Deep-Dive Resources:

■ cppreference — std::mutex — https://en.cppreference.com/w/cpp/thread/mutex
■ OSTEP — Locks chapter (free PDF) — https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf
■ Preshing on Programming — Memory Barriers — https://preshing.com/20120710/memory-barriers-are-like-source-control-operations/
■ The Cherno — Mutex & Thread Safety (YouTube) — https://www.youtube.com/watch?v=3ZxZPeXPaM4
■ Herb Sutter — atomic<> Weapons (CppCon, YouTube) — https://www.youtube.com/watch?v=c1gO9aB9nbs
■ ★ cppreference — std::shared_mutex (reader-writer lock) — https://en.cppreference.com/w/cpp/thread/shared_mutex

❖ Build Tasks:

- ❖ Reproduce a race condition with a shared counter — observe non-deterministic output
- ❖ Fix using std::mutex — confirm deterministic output
- ❖ Deliberately create a deadlock (two threads, two mutexes, opposite order)
- ❖ Fix with lock ordering (always lock in same global order)
- ❖ ★ Protect the DNS LRU cache with std::shared_mutex — multiple reader threads resolve simultaneously, writer upgrades lock only on cache miss

■ **Your URL queue will use mutex. Get race conditions and deadlocks right here.**

Day 11 — Condition Variables + Semaphores

Condition variables allow threads to sleep until work is available — essential for your thread pool. Without them, you'd busy-wait and burn 100% CPU.

Core Concepts:

- Spurious wakeups — always use while loop, never if
- Producer-consumer pattern — the classic bounded buffer problem
- std::condition_variable — wait(), notify_one(), notify_all()

- Semaphore concept — counting (capacity) vs binary (mutex-like)
- std::counting_semaphore (C++20) — cleaner than condition_variable for counting
- ★ Back-pressure in the crawler: if the Kafka queue is full, the URL frontier blocks — condition_variable enforces this

■ Deep-Dive Resources:

■ cppreference — std::condition_variable — https://en.cppreference.com/w/cpp/thread/condition_variable
■ OSTEP — Condition Variables (free PDF) — https://pages.cs.wisc.edu/~remzi/OSTEP/threads-cv.pdf
■ Bo Qian — C++ Threading: Condition Variable (YouTube) — https://www.youtube.com/watch?v=13dFggo4t_I
■ cppreference — std::counting_semaphore (C++20) — https://en.cppreference.com/w/cpp/thread/counting_semaphore
■ Herb Sutter — Mutex, Semaphore, Lock (Dr Dobbs) — https://www.drdobbs.com/cpp/mutex-and-semaphore-clarified/211201163
■ ★ Blueprint thread_pool.hpp — condition_variable usage in worker loop — distributed_search_engine_blueprint.pdf

❖ Build Tasks:

- ❖ Build producer-consumer queue with condition_variable
- ❖ Test with 4 producers and 2 consumers — print events
- ❖ Demonstrate spurious wakeup bug (using if) vs fix (using while)
- ❖ ★ Add a bounded capacity (max 1000 items) to the queue; producer blocks when full — simulate Kafka back-pressure

■ **This is the heart of your thread pool. Understand it deeply.**

Days 12–13 — Thread Pool

A thread pool reuses threads instead of spawning/destroying — critical for crawler performance. Spawning a thread for every URL fetch is 100-1000x slower.

Core Concepts:

- Fixed-size thread pool design — N worker threads running forever
- Task queue — std::function + std::queue protected by mutex
- Worker thread loop: wait on cv → dequeue → execute → repeat
- Graceful shutdown — stop flag + notify_all() to wake all workers
- Work stealing (advanced) — per-thread deques, steal from the back
- ★ Blueprint ThreadPool: 64 workers, Kafka producer callback per task — replicate this exact design

■ Deep-Dive Resources:

■ Jakob Progsch — ThreadPool (canonical C++ GitHub) — https://github.com/progschj/ThreadPool
■ YouTube — Build a Thread Pool from Scratch (CodeVault) — https://www.youtube.com/watch?v=6re5U82KwbY
■ Marius Bancila — Thread Pool in Modern C++ (blog) — https://mariusbancila.ro/blog/2019/11/13/a-thread-safe-queue-implementation/
■ OSTEP — Common Concurrency Bugs (free PDF) — https://pages.cs.wisc.edu/~remzi/OSTEP/threads-bugs.pdf
■ ★ Blueprint thread_pool.cpp — full constructor + enqueue implementation — distributed_search_engine_blueprint.pdf
■ ★ CppCon 2015 — C++ Atomics, From Basic to Advanced (YouTube) — https://www.youtube.com/watch?v=ZQFzMfHIxng

❖ Build Tasks:

- ❖ Build thread pool from scratch — no library
- ❖ Submit 1000 simulated fetch tasks — measure throughput vs single-thread
- ❖ Add graceful shutdown: join all workers cleanly
- ❖ ★ Add a future/promise return from enqueue so callers can wait on individual task results
- ❖ ★ Scale to 64 threads (blueprint spec) and benchmark on 10K simulated URL fetches — measure tasks/sec

■ **Week 2 Output: Production-grade thread pool with task queue and graceful shutdown.**

WEEK 3 — Networking: TCP/IP, Sockets, HTTP

Days 15–16 — TCP/IP Model

Every HTTP request your crawler makes travels through these layers. Know what each layer does and where failures can occur.

Core Concepts:

- OSI vs TCP/IP layers — application, transport, internet, link
- IP: addressing (IPv4/IPv6), routing, TTL, fragmentation, ICMP
- TCP: 3-way handshake (SYN/SYN-ACK/ACK), sequence numbers, ACK, sliding window
- UDP vs TCP tradeoffs — ordering, reliability, congestion control
- Ports, sockets, ephemeral ports, and NAT
- ★ TIME_WAIT state — why crawler sockets linger and how SO_REUSEADDR fixes it

■ Deep-Dive Resources:

■ GeeksforGeeks — TCP/IP Model (complete) — https://www.geeksforgeeks.org/tcp-ip-model/
■ Julia Evans — Networking Zine (free PDF) — https://wizardzines.com/zines/networking/
■ Computer Networks — Tanenbaum (free chapters) — https://csc-knu.github.io/sys-prog/books/Andrew%20S.%20Tanenbaum%20-%20Computer%20Networks.pdf
■ tcpdump tutorial — Daniel Miessler — https://danielmiessler.com/study/tcpdump/
■ Wireshark — Official Getting Started Guide — https://www.wireshark.org/docs/wsug_html_chunked/ChapterIntroduction.html
■ ★ Beej's Guide to Network Programming — Chapter 2: What is a socket? — https://beej.us/guide/bgnet/html/#what-is-a-socket

❖ Build Tasks:

- ❖ Trace an HTTP request with tcpdump — save pcap file
- ❖ Open in Wireshark — identify SYN, SYN-ACK, ACK, GET, 200 OK packets
- ❖ Identify the TCP three-way handshake and sequence numbers
- ❖ ★ Set SO_REUSEADDR on a server socket; restart it immediately after killing — observe that binding succeeds without TIME_WAIT delay

Days 17–18 — Socket Programming (MUST MASTER)

Sockets are the C API for network communication. Your crawler uses raw sockets under the hood. This is the most important topic in Week 3.

Core Concepts:

- socket() → bind() → listen() → accept() — server flow
- connect() for client side — TCP handshake triggered here
- send() / recv() and partial reads — recv returns up to N bytes
- Non-blocking sockets — O_NONBLOCK + select/poll/epoll
- Socket options: SO_REUSEADDR, TCP_NODELAY, SO_TIMEOUT
- Everything is a file descriptor — UNIX philosophy
- ★ Blueprint HTTPFetcher pattern: non-blocking connect() + select() timeout — implement exactly this

■ Deep-Dive Resources:

■ Beej's Guide to Network Programming (free online) — https://beej.us/guide/bgnet/html/
■ IBM — Socket programming fundamentals (classic tutorial) — https://www.ibm.com/docs/en/i/7.4?topic=communications-socket-programming

■ Linux man page — socket(7) — https://man7.org/linux/man-pages/man7/socket.7.html
■ epoll tutorial — Eli Bendersky — https://eli.thegreenplace.net/2017/concurrent-servers-part-3-event-driven/
■ CS144 Stanford — sockets lab (free) — https://cs144.github.io/
■ ★ Blueprint http_fetcher.cpp — full raw socket HTTP fetch with timeout — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Build TCP echo server (server receives, sends back)
- ✦ Build TCP client that sends raw HTTP GET — parse response manually
- ✦ Handle partial recv() correctly — loop until full response received
- ✦ Implement non-blocking connect() with select() timeout
- ✦ ★ Replicate blueprint HTTPFetcher::fetch() step-by-step — DNS resolve, socket, non-blocking connect, send request, recv loop, close

■ **Read Beej's Guide chapters 1–7 cover to cover. This week decides your networking career.**

Day 19 — DNS Resolution

Before connecting to any server, your crawler resolves hostnames. Every crawl starts here. Understand the full DNS resolution chain.

Core Concepts:

- DNS hierarchy: root (.) → TLD (.com) → authoritative → resolver
- Record types: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail)
- TTL and DNS caching — your LRU cache from Day 7 applies here
- getaddrinfo() API — thread-safe, returns linked list of addrinfo
- /etc/hosts and /etc/resolv.conf — local overrides
- ★ Blueprint DNSResolver: LRU + 300s TTL — implement this exact struct

■ Deep-Dive Resources:

■ Linux man page — getaddrinfo(3) — https://man7.org/linux/man-pages/man3/getaddrinfo.3.html
■ How DNS Works — dnsimple visual guide — https://howdns.works/
■ Julia Evans — DNS Zine (free) — https://wizardzines.com/zines/dns/
■ Cloudflare — What is DNS? (blog) — https://www.cloudflare.com/learning/dns/what-is-dns/
■ DNS over HTTPS explained — Mozilla blog — https://hacks.mozilla.org/2018/05/a-cartoon-intro-to-dns-over-https/
■ ★ Blueprint dns_resolver.hpp / dns_resolver.cpp — full implementation listing — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Implement DNS resolver using getaddrinfo() in C++
- ✦ Cache resolved IPs in your LRU cache with TTL expiry
- ✦ Measure: DNS lookup latency before and after cache hit
- ✦ ★ Implement blueprint DNSResolver exactly: unordered_map, mutex, 300s TTL; test with 20 distinct hostnames and confirm cache hit rate

Day 20 — HTTP Deep Dive

HTTP is the language your crawler speaks. Know the wire format exactly — every byte of the request and response.

Core Concepts:

- HTTP/1.1 request/response structure — start-line, headers, blank line, body
- Headers: Host, Content-Type, User-Agent, Accept-Encoding, Connection
- Status codes: 200, 301/302 (redirect), 404, 429 (rate limit), 503

• Keep-alive and Connection header — reuse TCP connections
• Chunked transfer encoding — streaming responses of unknown length
• HTTP/2 framing concept — multiplexing, HPACK header compression
• ★ User-Agent: 'IndiaBot/1.0' as in blueprint — always identify your crawler honestly

■ Deep-Dive Resources:

■ MDN Web Docs — HTTP Messages (authoritative) — https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages
■ HTTP/1.1 RFC 7230 (official spec) — https://datatracker.ietf.org/doc/html/rfc7230
■ High Performance Browser Networking — Ilya Grigorik (free book) — https://hpbn.co/
■ HTTP Status Codes — MDN reference — https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
■ Wireshark — Filtering HTTP traffic tutorial — https://www.wireshark.org/docs/dfref/h/http.html
■ ★ Blueprint http_fetcher.cpp — hand-crafted HTTP/1.1 GET with Host + User-Agent headers — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

✦ Parse raw HTTP response manually (no library) — extract status, headers, body
✦ Handle 301 redirect chains up to depth 5
✦ Implement basic gzip decompression (zlib/inflate)
✦ ★ Reproduce blueprint request string exactly: 'GET {path} HTTP/1.1\r\nHost: {host}\r\nUser-Agent: IndiaBot/1.0\r\nConnection: close\r\n\r\n'

Day 21 — Mini Project: HTTP Client in C++

Combine DNS resolution + TCP socket + HTTP protocol into a complete HTTP client. This becomes the core of your crawler's fetch engine.

Core Concepts:

• Connection pooling concept — reuse sockets across requests to same host
• Timeout handling with select() — don't block forever on slow servers
• Error handling: ECONNREFUSED, ETIMEDOUT, EHOSTUNREACH
• SSL/TLS concept — OpenSSL SNI for HTTPS
• ★ Blueprint HTTPFetcher::fetch() with 5000ms timeout — this is your exact target implementation

■ Deep-Dive Resources:

■ Beej's Guide — Client-Server examples — https://beej.us/guide/bgnet/html/#client-server-background
■ OpenSSL — Simple TLS client example (GitHub) — https://github.com/openssl/openssl/blob/master/demos/sslecho/main.c
■ libcurl internals — how curl works under the hood — https://curl.se/dev/internals.html
■ CS144 Stanford Lab 0 — writing a webget — https://cs144.github.io/
■ ★ Advanced checklist item: HTTPS with OpenSSL SNI — see blueprint Chapter 11 — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

✦ Fetch https://example.com and print status code + first 500 bytes of body
✦ Handle redirects up to depth 5 automatically
✦ Measure latency of 100 sequential requests to same host (keep-alive)
✦ ★ Add OpenSSL TLS wrapping — fetch an HTTPS URL end-to-end through your raw socket client

■ Week 3 Output: C++ HTTP client with DNS resolution, redirect handling, connection reuse.

WEEK 4 — Crawler: Full Implementation

Day 22 — URL Parsing + Normalisation

URLs look simple but have dozens of edge cases. Normalise aggressively — duplicate URLs waste bandwidth and pollute your index.

Core Concepts:

- RFC 3986 URL structure: scheme, authority, path, query, fragment
- Percent-encoding — %20 = space, must decode before comparison
- Relative URL resolution — base URL + ../relative → absolute
- Normalisation: lowercase scheme/host, remove fragment, sort query params
- Canonicalisation: remove tracking params (utm_source, fbclid, etc.)
- ★ Per-domain crawl budget enforcement requires normalised domain extraction

■ Deep-Dive Resources:

■ RFC 3986 — URI Generic Syntax (official) — https://datatracker.ietf.org/doc/html/rfc3986
■ Ada URL Parser (fast, WHATWG-compliant, C++) — https://github.com/ada-url/ada
■ Google URL Normalization blog — https://developers.google.com/search/docs/crawling-indexing/canonicalization
■ URI parsing in C++ — uriparser library — https://uriparser.github.io/
■ URL Fuzzer — testing URL edge cases — https://claroty.com/team82/blog/fuzzing-url-parsers
■ ★ Blueprint url_frontier.hpp — domain extraction from normalised URL — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Parse 20 malformed URLs correctly (missing scheme, relative paths, encoded chars)
- ✦ Resolve relative URLs against base URL — handle ../../../ paths
- ✦ Strip tracking parameters (utm_*, fbclid, gclid) from 1000 URLs
- ✦ ★ Implement a domain extractor; group 10K crawled URLs by domain; enforce max 500 pages/domain budget

Day 23 — robots.txt + Politeness

Ethical crawling requires robots.txt compliance and rate limiting. Getting blocked = losing all your crawl progress.

Core Concepts:

- robots.txt syntax: User-agent, Disallow, Allow, Crawl-delay, Sitemap
- Wildcard matching: * and $ in path patterns
- Per-host rate limiting with token bucket algorithm
- 429 Too Many Requests — exponential backoff with jitter
- Sitemap.xml discovery — seed URL expansion
- ★ Blueprint robots_checker.hpp — per-domain delay map protected by shared_mutex

■ Deep-Dive Resources:

■ Google — robots.txt specification — https://developers.google.com/search/docs/crawling-indexing/robots/intro
■ robots.txt spec — robotstxt.org — https://www.robotstxt.org/robotstxt.html
■ Token Bucket Algorithm — Wikipedia — https://en.wikipedia.org/wiki/Token_bucket
■ Exponential backoff — Google Cloud best practices — https://cloud.google.com/iot/docs/how-tos/exponential-backoff
■ Sitemap protocol — sitemaps.org — https://www.sitemaps.org/protocol.html
■ ★ Blueprint Chapter 2.5 — Politeness: 1s per-domain delay, Bloom Filter for dedup — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Parse robots.txt for 5 real websites — print allowed/disallowed paths
- ✦ Implement per-host rate limiter with token bucket algorithm
- ✦ Test exponential backoff on simulated 429 responses
- ✦ ★ Wire politeness into thread pool: per-domain timestamp map (shared_mutex); block enqueue until delay elapsed

Days 24–25 — Multithreaded Crawler Architecture

Combine thread pool + URL queue + HTTP client into a scalable crawler. This is the centrepiece of the first half of the project.

Core Concepts:

- Frontier queue: FIFO vs priority (prioritize by PageRank estimate)
- Per-host queues — isolate requests per domain for politeness
- URL scheduling: BFS (broad coverage) vs DFS vs best-first
- Back-pressure — don't enqueue faster than you can crawl
- Crawl depth limits — prevent infinite loops in circular link graphs
- ★ Blueprint main.cpp loop: BLPOP from Redis → Bloom check → pool.enqueue() → Kafka send

■ Deep-Dive Resources:

■ Web Crawling Architecture — CommonCrawl blog — https://commoncrawl.org/the-data/how-it-works/
■ Mercator Crawler paper (seminal web crawler design) — https://research.google/archive/mercator.html
■ Scrapy architecture (Python reference implementation) — https://docs.scrapy.org/en/latest/topics/architecture.html
■ Redis BLPOP — blocking list pop for URL queue — https://redis.io/commands/blpop/
■ pclub.in — Systems Roadmap — https://pclub.in/roadmaps
■ ★ Blueprint main.cpp — full crawler loop with Bloom filter + Kafka producer — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Crawler fetches 100 URLs with 8 threads simultaneously
- ✦ Per-host rate limiting active — max 1 req/s per domain
- ✦ Saves HTML + metadata JSON to disk
- ✦ ★ Replace disk queue with Redis BLPOP frontier; send fetched HTML to a mock Kafka topic (confluent-kafka-python)

Day 26 — Bloom Filter for URL Deduplication

A seen-URL set can grow to billions of entries. Bloom filters give O(1) membership test with ~2GB of memory instead of 200GB for a HashSet at 1B URLs.

Core Concepts:

- Bloom filter: bit array + k hash functions, O(k) insert/query
- False positive rate formula: (1 - e^(-kn/m))^k
- Choosing k and m for target FPR — 0.1% FPR at 1B URLs needs ~2GB
- Counting Bloom filter — supports deletions (4-bit counters)
- Scalable Bloom filter — chains multiple filters for unbounded growth
- ★ Blueprint spec: murmur3 hash, 2GB bit-array, ~0.1% FPR at 1B URLs

■ Deep-Dive Resources:

■ Bloom Filters by Example — visual — https://llimllib.github.io/bloomfilter-tutorial/
■ Bloom filter optimal params calculator — https://hur.st/bloomfilter/
■ Original Bloom filter paper (1970) — Burton Bloom — https://dl.acm.org/doi/10.1145/362686.362692

■ RedisBloom — Bloom filter for distributed crawlers — https://redis.io/docs/data-types/probabilistic/bloom-filter/
■ Murmur3 hash — fast non-cryptographic hash (GitHub) — https://github.com/aappleby/smhasher
■ ★ Blueprint Chapter 2.5 — Bloom Filter: murmur3, 2GB bit-array, 0.1% FPR at 1B URLs — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Implement Bloom filter: insert + query (no library)
- ✦ Measure FPR empirically vs theoretical at 100K, 500K, 1M inserts
- ✦ Compare memory: Bloom filter vs std::unordered_set at 1M URLs
- ✦ ★ Use RedisBloom BF.ADD / BF.EXISTS to replicate blueprint's distributed dedup; test with 1M URLs

Days 27–28 — Full Crawler Integration

Wire everything together. Add persistence, error handling, duplicate content detection, and crawl statistics. This is your first deployable component.

Core Concepts:

- • Checkpoint + resume — save frontier to Redis/disk after every 1000 pages
- • Crawl stats: pages/sec, errors, redirects, bytes fetched
- • Duplicate content detection — MD5/SHA256 of response body
- • Link extraction — parse anchor href tags with lxml/BeautifulSoup
- • Per-domain crawl budget — max N pages per domain
- • ★ Output format: Kafka message with (url, sha256, html) as per blueprint pipeline contract

■ Deep-Dive Resources:

■ CommonCrawl — how they handle scale — https://commoncrawl.org/
■ lxml — Fast HTML/XML parser for link extraction — https://lxml.de/parsing.html
■ hashlib — MD5/SHA256 in Python for content dedup — https://docs.python.org/3/library/hashlib.html
■ GitHub — heritrix3 (Internet Archive's production crawler) — https://github.com/internetarchive/heritrix3
■ Scrapy — checkpoint/resume documentation — https://docs.scrapy.org/en/latest/topics/jobs.html
■ ★ Blueprint docker-compose.yml — crawler service with replicas: 4 — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Crawl 500+ pages starting from seed URLs
- ✦ Checkpoint and resume after intentional interrupt
- ✦ Output: per-page JSON with url, title, links[], timestamp, content_hash
- ✦ ★ Publish each crawled page to Kafka 'raw_pages' topic; verify consumer receives all messages
- ✦ ★ Run 4 crawler replicas via docker-compose scale — confirm no URL duplication via Bloom filter

■ Week 4 Output: Multithreaded C++ crawler: 500+ pages, dedup, rate-limiting, checkpointing.

WEEK 5 — NLP Basics: Search Pipeline

Day 29 — Text Preprocessing

Raw HTML is 70-90% noise. Aggressive cleaning before any NLP dramatically improves index quality and reduces storage by 5-10x.

Core Concepts:

- HTML tag stripping vs structured parsing — strip removes everything, parsing is selective
- Unicode normalisation: NFC (canonical composition) vs NFD (decomposition)
- Whitespace normalisation — collapse tabs, newlines, multiple spaces
- Boilerplate removal: nav, footer, ads, cookie banners — heuristic text density
- Character encoding detection — chardet / charset-normalizer
- ★ Blueprint NOISE_TAGS: script, style, nav, footer, header, aside, form — exact list to decompose

■ Deep-Dive Resources:

■ NLTK Book — Chapter 3: Processing Raw Text (free online) — https://www.nltk.org/book/ch03.html
■ BeautifulSoup Documentation — HTML parser — https://www.crummy.com/software/BeautifulSoup/bs4/doc/
■ lxml — Fast HTML processing — https://lxml.de/
■ chardet — Encoding detection library — https://chardet.readthedocs.io/en/latest/
■ Readability.js — Mozilla's boilerplate removal algorithm — https://github.com/mozilla/readability
■ ★ Blueprint html_parser.py — extract_text() with title, desc, body, h1s, h2s — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Strip HTML from 100 crawled pages, measure text yield
- ✦ Normalise whitespace and Unicode (NFC) across all pages
- ✦ Remove boilerplate with text-density heuristic (>50 words per block)
- ✦ ★ Implement blueprint extract_text() exactly: return dict with title, description, body, h1, h2 — verify on 50 real pages

Day 30 — Tokenisation + Stemming + Lemmatisation

Break text into tokens and reduce inflected forms. This affects recall dramatically — 'running' and 'runs' should match 'run'.

Core Concepts:

- Word tokenisation: whitespace vs regex vs BPE (byte-pair encoding)
- Sentence tokenisation — Punkt tokenizer for Indian languages too
- Stemming (Porter, Snowball): fast, lossy — 'running' → 'run'
- Lemmatisation (WordNet): accurate, slow — 'ran' → 'run'
- Stop word lists — when NOT to remove (e.g. 'to be or not to be')
- ★ Blueprint tokenizer.py: SnowballStemmer for English, Stanza lemmatizer for Indian langs

■ Deep-Dive Resources:

■ NLTK — tokenization documentation — https://www.nltk.org/api/nltk.tokenize.html
■ spaCy — Tokenization (industrial strength) — https://spacy.io/usage/linguistic-features#tokenization
■ Snowball — Porter stemmer (original paper + impl) — https://snowballstem.org/
■ Stanford NLP — NLTK vs spaCy comparison — https://www.analyticsvidhya.com/blog/2021/10/nltk-vs-spacy/
■ Real Python — spaCy NLP tutorial — https://realpython.com/natural-language-processing-spacy-python/
■ ★ Blueprint tokenizer.py — full STANZA_LANGS set + tokenize() dispatch — distributed_search_engine_blueprint.pdf

✥ Build Tasks:

- ✥ Tokenise + stem 10,000 documents with NLTK Snowball stemmer
- ✥ Compare Porter vs Snowball output on 100 sample words
- ✥ Build bigram index alongside unigram — measure index size difference
- ✥ ★ Implement blueprint tokenize() with lang dispatch: English → Snowball, Indian langs → Stanza lemma, fallback → indic_tokenize

Day 31 — TF-IDF

TF-IDF is the baseline ranking signal. Understand the math before implementing BM25 — BM25 is essentially TF-IDF with better normalisation.

Core Concepts:

- TF: raw count vs log-normalised (1 + log tf) vs augmented TF
- IDF: log(N/df) — penalises common terms, rewards rare terms
- Smooth IDF: log(N+1/df+1) + 1 — avoids zero IDF for terms in all docs
- TF-IDF vector space model — documents as vectors in term space
- Cosine similarity — magnitude-normalised dot product
- ★ Field weighting preview: title TF-IDF weight 3.0, body 1.0 — motivates BM25F in Week 7

■ Deep-Dive Resources:

■ scikit-learn — TfidfVectorizer documentation — https://scikit-learn.org/stable/modules/feature_extraction.html#tfidf-term-weighting
■ Stanford IR Book — Chapter 6: Scoring (free online) — https://nlp.stanford.edu/IR-book/html/htmledition/tf-idf-weighting-1.html
■ TF-IDF from scratch in Python — Towards Data Science — https://towardsdatascience.com/tf-idf-for-document-ranking-from-scratch-in-python-on-real-world-dataset-796d339a4089
■ BM25 intro — ethen8181 GitHub (math + Python) — https://ethen8181.github.io/machine-learning/search/bm25_intro.html
■ Information Retrieval — Manning, Raghavan, Schutze (free PDF) — https://nlp.stanford.edu/IR-book/
■ ★ Blueprint field weights: title=3.0, h1/h2=2.0, body=1.0, URL=1.5, desc=1.2 — distributed_search_engine_blueprint.pdf

✥ Build Tasks:

- ✥ Compute TF-IDF for 1000 documents using scikit-learn
- ✥ Find top-5 similar documents to a query using cosine similarity
- ✥ Implement TF-IDF from scratch (no sklearn) — compare outputs
- ✥ ★ Apply blueprint field weights to TF-IDF: score = 3.0*TF_title + 1.0*TF_body; compare ranking vs unweighted

Days 32–35 — Full NLP Pipeline + Language Detection

Build the complete document processing pipeline. This is your data ETL: raw HTML in → structured, searchable JSON out.

Core Concepts:

- FastText language ID — 176 languages, character n-gram based, very fast
- Content quality signals: text density (words / tags), link density
- Near-duplicate detection: shingling + MinHash — detect 90% similar pages
- Named entity recognition (spaCy) — PERSON, ORG, LOC, GPE tags
- Keyword extraction — YAKE, RAKE — unsupervised, no training needed
- ★ Blueprint detect_lang(): fastText lid.176.bin, confidence > 0.7, else 'unknown'
- ★ Blueprint NER: spaCy en_core_web_sm for English; Stanza NER for Hindi/Tamil/etc.

■ Deep-Dive Resources:

■ fastText — Language Identification model (free download) — https://fasttext.cc/docs/en/language-identification.html
■ spaCy 101 — NLP pipeline documentation — https://spacy.io/usage/spacy-101
■ YAKE — Keyword Extraction (paper + Python) — https://github.com/LIAAD/yake
■ MinHash for near-duplicate detection — Mining Massive Datasets — http://www.mmds.org/
■ NLTK Book — Chapter 7: Information Extraction (free) — https://www.nltk.org/book/ch07.html
■ ★ Blueprint pipeline/ directory — consumer.py, lang_detector.py, ner.py, index_emitter.py — distributed_search_engine_blueprint.pdf
■ ★ Apache Kafka Python — confluent-kafka consumer docs — https://docs.confluent.io/kafka-clients/python/current/overview.html

❖ Build Tasks:

- ❖ Process 1000 HTML pages through full pipeline
- ❖ Output: {url, title, tokens, lang, keywords, entities, timestamp}
- ❖ Filter: remove pages with <50 words or detected as non-English/Hindi
- ❖ ★ Build a Kafka consumer (consumer.py) that reads 'raw_pages', runs the full pipeline, and writes structured JSON to a 'processed_pages' topic
- ❖ ★ Implement blueprint index_emitter.py: emit (term, lang, doc_id, tf, positions[]) tuples to the indexer

■ **Week 5 Output: Python NLP pipeline: HTML → structured JSON with tokens, entities, language.**

WEEK 6 — Database Systems

Days 36–37 — PostgreSQL + Indexing

Postgres stores your document metadata. Understanding how it actually executes queries — and how indexes change execution plans — is what separates seniors from juniors.

Core Concepts:

- B-tree index: structure, range queries, uniqueness, composite ordering
- Hash index: equality-only, faster than B-tree for =, not for BETWEEN
- GIN index: inverted index for full-text search, JSONB, arrays
- EXPLAIN ANALYZE: reading query plans — Seq Scan vs Index Scan vs Index Only Scan
- VACUUM, autovacuum, dead tuples — why UPDATE/DELETE create bloat
- ★ Blueprint schema: urls, links, entities, query_log tables — implement all four

■ Deep-Dive Resources:

■ PostgreSQL Official Tutorial — https://www.postgresql.org/docs/current/tutorial.html
■ use-the-index-luke.com — SQL indexing for developers (free book) — https://use-the-index-luke.com/
■ CMU 15-445 — Database Systems (free lectures, YouTube 2024) — https://15445.courses.cs.cmu.edu/fall2024/
■ PostgreSQL EXPLAIN visualizer — explain.dalibo.com — https://explain.dalibo.com/
■ Hussein Nasser — PostgreSQL Indexing deep dive (YouTube) — https://www.youtube.com/watch?v=fsG1XaZEa78
■ ★ Blueprint Chapter 4.2 — Full PostgreSQL schema with indexes — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Schema: documents(id, url, title, lang, crawled_at, word_count)
- ✦ Add indexes on url (unique), lang, crawled_at
- ✦ EXPLAIN ANALYZE a query filtering by lang AND date range — compare before/after index
- ✦ ★ Implement full blueprint schema: urls (with pagerank), links, entities, query_log; add all listed indexes including idx_urls_rank (pagerank DESC)
- ✦ ★ Write a JOIN query: top-10 English URLs by PageRank crawled in the last 7 days — verify Index Scan on idx_urls_rank

Days 38–39 — Redis + RocksDB

Redis is your high-speed ephemeral layer; RocksDB is your persistent inverted index store. Understanding when to use each is a system design skill.

Core Concepts:

- Redis data types: String (DNS cache), List (URL queue), Sorted Set (priority), Hash
- LPUSH/RPOP (FIFO queue), BLPOP (blocking pop for workers)
- RocksDB LSM-tree: MemTable → SSTable → Compaction
- Write-ahead log (WAL) for durability, Bloom filter per SSTable
- LSM vs B-tree tradeoffs: LSM better for writes, B-tree for point reads
- ★ Blueprint posting list key format: b'p:{shard_id}:{term}' → zstd-compressed delta-encoded bytes

■ Deep-Dive Resources:

■ Redis Documentation — Data Types (official) — https://redis.io/docs/latest/develop/data-types/
■ Redis University — RU101 Introduction to Redis (free) — https://redis.io/university/
■ RocksDB wiki — getting started — https://github.com/facebook/rocksdb/wiki

■ RocksDB internals — Mark Callaghan blog — http://smalldatum.blogspot.com/
■ CMU 15-445 — Storage lecture (YouTube) — https://www.youtube.com/watch?v=DJ5u5HrbcMk
■ ★ Blueprint Chapter 4.3 — RocksDB encode_postings() + zstd compression — distributed_search_engine_blueprint.pdf

❖ Build Tasks:

- ❖ Implement URL frontier in Redis List (LPUSH seeds, BLPOP workers)
- ❖ DNS cache with TTL using SET EX
- ❖ Store 100K term→postings mappings in RocksDB, benchmark read latency
- ❖ ★ Implement blueprint encode_postings() with delta encoding + struct.pack; apply zstd level-3 compression; measure compression ratio

Days 40–42 — Data Layer Integration

Wire Postgres + Redis + RocksDB into a coherent three-tier storage architecture. Each store has a clear responsibility.

Core Concepts:

- • Metadata (URL, title, lang, pagerank) → PostgreSQL (relational, ACID)
- • Inverted posting lists (term → docID, positions) → RocksDB (write-optimized)
- • URL queue + DNS cache + crawl stats → Redis (in-memory, TTL)
- • Connection pooling — pgBouncer concept, SQLAlchemy pool
- • Write-through vs write-behind caching — consistency tradeoffs
- • ★ CAP position: Redis URL frontier is CP; Postgres + RocksDB are AP — accept stale entries, re-crawl on 404

■ Deep-Dive Resources:

■ SQLAlchemy — Connection Pool documentation — https://docs.sqlalchemy.org/en/20/core/pooling.html
■ pgBouncer — connection pooling for PostgreSQL — https://www.pgbouncer.org/
■ Redis + Python — redis-py documentation — https://redis-py.readthedocs.io/en/stable/
■ python-rocksdb — RocksDB Python bindings — https://github.com/twmht/python-rocksdb
■ CAP Theorem explained — Martin Kleppmann — https://martin.kleppmann.com/2015/05/11/please-stop-calling-databases-cp-or-ap.html
■ ★ Blueprint Chapter 4.1 — full storage choice rationale table — distributed_search_engine_blueprint.pdf
■ ★ ClickHouse for query analytics — setup guide — https://clickhouse.com/docs/en/getting-started/quick-start

❖ Build Tasks:

- ❖ Store 10K crawled documents across all three systems
- ❖ Query: find all English docs crawled today with word_count > 500
- ❖ Measure write throughput for batch inserts into each system
- ❖ ★ Add ClickHouse as fourth analytics store; write query_log events to it; run a CTR-by-rank aggregation query

■ Week 6 Output: 3-tier storage — Postgres (metadata) + Redis (queue/cache) + RocksDB (index).

WEEK 7 — Information Retrieval: Core

Day 43 — Inverted Index

The inverted index is the data structure that makes search fast. Every search engine — from Google to Elasticsearch — is built on it.

Core Concepts:

- Inverted index: term → postings list (docID, tf, positions[])
- Dictionary: hash table or B-tree for term lookup — trade speed vs order
- Postings list: sorted by docID for merge efficiency
- Positional index: stores word positions for phrase queries ('new york')
- Skip pointers: jump over sections of long postings lists
- Compression: delta encoding (store diffs), variable byte encoding
- ★ Blueprint field weights applied at index time: title tokens added with weight 3.0 vs body tokens at 1.0

■ Deep-Dive Resources:

■ Stanford IR Book — Chapter 1: Boolean Retrieval (free) — https://nlp.stanford.edu/IR-book/html/htmledition/boolean-retrieval-1.html
■ Stanford IR Book — Chapter 2: Inverted Index (free) — https://nlp.stanford.edu/IR-book/html/htmledition/a-first-take-at-building-an-inverted-index-1.html
■ IR-Book PDF — Manning, Raghavan, Schutze (free) — https://nlp.stanford.edu/IR-book/
■ Medium — Building an Inverted Index from Scratch — https://medium.com/@fro_g/writing-a-simple-inverted-index-in-python-3c8bcb52169a
■ Tantivy — Rust search engine library (study the code) — https://github.com/quickwit-oss/tantivy
■ ★ Blueprint Chapter 5.1 — field weight table + posting list diagram — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Build inverted index for 10K documents in Python
- ✦ Support AND/OR queries via postings intersection/union
- ✦ Measure: index size vs raw text size, query latency
- ✦ ★ Add field weights during indexing: call add_document() once per field with its weight; verify 'india' in title scores higher than 'india' in body

Days 44–45 — Index Builder + Compression

A production inverted index uses SPIMI and compressed delta-encoded posting lists to reduce disk I/O 5-10x.

Core Concepts:

- SPIMI: single-pass in-memory indexing — process docs one at a time, flush partial indexes
- External merge sort — merge partial indexes without loading all into RAM
- Variable byte encoding: 1 byte per small int (< 128), up to 5 bytes for large
- Delta encoding: store doc[i] - doc[i-1] instead of absolute IDs
- Index partitioning: term-based (each shard owns a term range) vs doc-based
- ★ Blueprint SegmentWriter.flush() — WriteBatch to RocksDB; 8-shard consistent hash

■ Deep-Dive Resources:

■ Stanford IR Book — Chapter 4: Index Construction (free) — https://nlp.stanford.edu/IR-book/html/htmledition/index-construction-1.html

■ Stanford IR Book — Chapter 5: Index Compression (free) — https://nlp.stanford.edu/IR-book/html/htmledition/index-compression-1.html
■ Tantivy blog — How Tantivy indexes text — https://fulmicoton.com/posts/behold-tantivy-part-1/
■ zstd — Zstandard compression (Facebook, GitHub) — https://github.com/facebook/zstd
■ pisa — Performant Indexes and Search for Academia — https://github.com/pisa-engine/pisa
■ ★ Blueprint index_builder.py — SegmentWriter + flush() + get_shard() consistent hash — distributed_search_engine_blueprint.pdf

❖ Build Tasks:

- ❖ Index 50K documents using SPIMI approach — flush every 10K docs
- ❖ Compress postings with delta + VByte encoding — measure compression ratio
- ❖ Store final merged index in RocksDB
- ❖ ★ Implement blueprint get_shard() (MD5 mod 8); verify terms distribute evenly across 8 RocksDB instances
- ❖ ★ Benchmark query fan-out: broadcast query to all 8 shards, merge top-k results — measure latency

Day 46 — BM25 Ranking

BM25 is the industry standard retrieval function, used in Elasticsearch, Lucene, and virtually every modern search engine. Understand every parameter.

Core Concepts:

- • BM25 formula: TF saturation with k1, document length normalisation with b
- • k1 (1.2–2.0): controls how much repeated terms matter — higher = more TF weight
- • b (0.75): controls length normalisation — 0 disables, 1 fully normalises
- • IDF with smoothing: log((N - df + 0.5) / (df + 0.5) + 1) — no negative values
- • BM25+ variant: adds lower bound delta to fix zero scoring for matched terms
- • Field-based BM25F: different k1/b per field (title, body, anchor text)
- • ★ Blueprint BM25: K1=1.5, B=0.75; query_score() method — implement exactly this

■ Deep-Dive Resources:

■ Stanford IR Book — Okapi BM25 (free) — https://nlp.stanford.edu/IR-book/html/htmledition/okapi-bm25-1.html
■ Michael Brenndoerfer — BM25 Complete Guide (interactive blog) — https://mbrenndoerfer.com/writing/bm25-search-algorithm-elasticsearch-implementation
■ BM25 derivation tutorial — ethen8181 GitHub — https://ethen8181.github.io/machine-learning/search/bm25_intro.html
■ Elasticsearch BM25 tuning guide — https://www.elastic.co/guide/en/elasticsearch/reference/current/similarity.html
■ Stanford CS276 — Ranking handout (PDF) — https://web.stanford.edu/class/cs276/handouts/lecture12-bm25etc.pdf
■ ★ Blueprint Chapter 6.1 — BM25 class with idf() + score() + query_score() — distributed_search_engine_blueprint.pdf

❖ Build Tasks:

- ❖ Implement BM25 from scratch (no library) in Python
- ❖ Tune k1 and b on a sample query set — find best nDCG@10
- ❖ Compare BM25 vs TF-IDF recall@10 on same query set
- ❖ ★ Implement blueprint BM25 class exactly: K1=1.5, B=0.75; wire to RocksDB posting list reader

Day 47 — PageRank

Link structure reveals authority. PageRank uses the web graph to boost pages that many trusted pages link to — the original Google insight.

Core Concepts:

- • Random surfer model — at each page, follow a link or teleport randomly

- PageRank formula: PR(u) = (1-d)/N + d * SUM(PR(v)/L(v))
- Damping factor d=0.85 — teleportation probability
- Power iteration until convergence (typically 50 iterations)
- Dangling nodes — pages with no outlinks absorb rank; redistribute evenly
- TrustRank — seed trusted pages to combat link spam
- ★ Blueprint pagerank.py: numpy-based iterative computation on links table, writes back to urls.pagerank column

■ Deep-Dive Resources:

■ Original PageRank paper — Brin & Page 1998 (PDF) — http://ilpubs.stanford.edu:8090/422/1/1999-66.pdf
■ 3Blue1Brown — What is PageRank? (YouTube) — https://www.youtube.com/watch?v=H9nSg3rQ3zE
■ NetworkX — PageRank implementation (Python) — https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.link_analysis.pagerank_alg.pagerank.html
■ Stanford CS246 — Mining the Web (free slides) — http://web.stanford.edu/class/cs246/
■ Mining Massive Datasets — Chapter 5: Link Analysis (free book) — http://www.mmds.org/
■ ★ Blueprint Chapter 6.2 — compute_pagerank() with in_links dict + numpy array — distributed_search_engine_blueprint.pdf

❖ Build Tasks:

- ❖ Compute PageRank on your crawled link graph
- ❖ Identify top-10 most authoritative pages — verify they make sense
- ❖ Experiment with different damping factors (0.7, 0.85, 0.95) — compare rankings
- ❖ ★ Implement blueprint compute_pagerank() exactly: 50 iterations, damping=0.85, UPDATE urls SET pagerank via executemany

Days 48–49 — Query Engine + Result Ranking

The query engine is the front door to your search engine. It must be fast (<100ms), accurate, and handle spelling errors and phrasing variation.

Core Concepts:

- Query processing pipeline: tokenise → stem → lookup → score → rank
- DAAT (Document-At-A-Time) vs TAAT (Term-At-A-Time) query processing
- Top-k retrieval with min-heap — maintain top 10 without sorting all results
- Score combination: BM25 * 0.7 + PageRank * 0.3 (tune weights with nDCG)
- Snippet generation: find the best window of 40 words containing query terms
- Spell correction: edit distance + language model prior
- ★ Blueprint QueryEngine.search(): parse → expand (WordNet) → retrieve → rerank → snippet — implement all stages
- ★ Blueprint score fusion: 0.7 * bm25 + 0.3 * pagerank — this ratio is tunable via A/B test

■ Deep-Dive Resources:

■ Stanford IR Book — Chapter 7: Computing Scores (free) — https://nlp.stanford.edu/IR-book/html/htmledition/computing-scores-in-a-complete-search-system-1.html
■ SymSpell — 1 million times faster spelling correction — https://github.com/wolfgarbe/SymSpell
■ Elasticsearch Query DSL — study production query engine design — https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl.html
■ Lucene — how scoring works (Apache docs) — https://lucene.apache.org/core/9_7_0/core/org/apache/lucene/search/package-summary.html#package.description
■ pclub.in search roadmap — curated resources — https://pclub.in/roadmaps
■ ★ Blueprint Chapter 6.3 — QueryEngine.search() + generate_snippet() full implementation — distributed_search_engine_blueprint.pdf
■ ★ Blueprint Chapter 7.3 — Cross-lingual retrieval via Indic transliterator — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Query engine returns top-10 results in <100ms for 10K doc index
- ✦ Generate highlighted snippets for each result
- ✦ Combine BM25 + PageRank scores — compare ranking quality with/without PageRank
- ✦ ★ Implement blueprint generate_snippet(): best 40-word window containing query terms
- ✦ ★ Add cross-lingual retrieval: detect Roman-script Hindi queries, transliterate to Devanagari, merge results from both shards

■ **Week 7 Output: Search engine: inverted index + BM25 + PageRank, <100ms query latency.**

WEEKS 8–9 — Backend (FastAPI) + Frontend

Days 50–52 — FastAPI Search API

FastAPI is async, auto-documented, and type-safe. Expose all search features through a clean REST API with pagination, caching, and rate limiting.

Core Concepts:

- Async/await in Python — event loop, coroutines, asyncio
- Path params, query params, request body — Pydantic models for validation
- GET /search?q=&page;= — paginated results with Redis cache
- POST /crawl — trigger crawl job as background task
- Rate limiting with slowapi — 100 req/min per IP
- OpenAPI auto-documentation — /docs endpoint (Swagger UI)
- ★ Blueprint main.py: /search, /autocomplete endpoints; Redis cache TTL=300s; latency logging

■ Deep-Dive Resources:

■ FastAPI Official Tutorial (best docs in Python ecosystem) — https://fastapi.tiangolo.com/tutorial/
■ Real Python — FastAPI Tutorial — https://realpython.com/fastapi-python-web-apis/
■ FastAPI + Redis caching — Towards Data Science — https://towardsdatascience.com/redis-caching-in-fastapi-a-practical-guide-with-code-examples-61dc7fd42695
■ slowapi — Rate Limiting for FastAPI — https://github.com/laurentS/slowapi
■ Locust — Load testing tool for Python — https://locust.io/
■ ★ Blueprint Chapter 9.2 — main.py with full /search + /autocomplete + latency tracking — distributed_search_engine_blueprint.pdf

❖ Build Tasks:

- ❖ Build /search endpoint with pagination (10 results/page)
- ❖ Cache repeated queries in Redis for 60 seconds
- ❖ Load test: sustain 100 req/s with locust — profile bottlenecks
- ❖ ★ Implement blueprint /autocomplete endpoint: engine.suggest(q, limit=8) via PostgreSQL trigram index (pg_trgm extension)
- ❖ ★ Add /api/click endpoint: receives {session_id, query, clicked_pos, doc_id} and writes to query_log

■ Week 8 Output: FastAPI search API with pagination, Redis caching, and rate limiting.

Days 53–56 — Search UI (Jinja2 + HTML/CSS)

Build a clean, functional search interface with result snippets, term highlighting, pagination, and basic click tracking.

Core Concepts:

- Jinja2 template inheritance — base.html + blocks pattern
- Search results page: rank, title, URL, snippet, score
- Term highlighting in snippets — wrap query terms in tags
- Pagination with prev/next links — correct boundary conditions
- HTMX for partial page updates — search-as-you-type without full React
- ★ Blueprint results.html: entity tag display, click tracking JS (trackClick), language selector

■ Deep-Dive Resources:

■ Jinja2 Documentation — Template Designer Guide — https://jinja.palletsprojects.com/en/3.1.x/templates/
■ MDN — HTML Semantics Guide — https://developer.mozilla.org/en-US/docs/Learn/HTML
■ HTMX Documentation — zero-JS interactivity — https://htmx.org/docs/
■ CSS Flexbox guide — css-tricks.com (definitive) — https://css-tricks.com/snippets/css/a-guide-to-flexbox/
■ Google Search result page — inspect the HTML structure — https://www.google.com/
■ ★ Blueprint Chapter 9.3 — results.html with entity tags + click tracking JS beacon — distributed_search_engine_blueprint.pdf

❖ Build Tasks:

- ❖ Search results page with snippets + query term highlighting
- ❖ Pagination for 10 results per page with prev/next
- ❖ Mobile-responsive layout using CSS flexbox/grid
- ❖ Basic click tracking: log (session_id, query, clicked_rank) to Postgres
- ❖ ★ Implement blueprint results.html: language selector (auto/en/hi/ta), entity tag badges, trackClick() JS beacon
- ❖ ★ Add dwell time measurement: record timestamp on click, send elapsed time on page unload to /api/dwell

■ **Week 9 Output: Search UI — query box, ranked results, snippets, pagination, click tracking.**

WEEK 10 — Deployment: Docker + Nginx

Days 57–60 — Full Stack Deployment

Package every service into Docker containers, orchestrate with Compose, put Nginx in front, and deploy to a free VPS. Your search engine goes live.

Core Concepts:

- Dockerfile: FROM, RUN, COPY, CMD, EXPOSE — layer caching strategy
- Multi-stage builds — compile C++ in builder stage, copy binary to slim runtime
- docker-compose.yml — services, networks, volumes, depends_on, healthcheck
- Nginx: reverse proxy (proxy_pass), SSL termination (Let's Encrypt), gzip, rate limiting
- Prometheus + Grafana concept — metrics and dashboards
- Structured logging (JSON format) — grep-friendly, ingestible by ELK
- ★ Blueprint docker-compose.yml: crawler (4 replicas), pipeline (2 replicas), backend, nginx, postgres, redis, kafka

■ Deep-Dive Resources:

■ Docker Official Get Started Tutorial — https://docs.docker.com/get-started/
■ Docker Compose Getting Started — https://docs.docker.com/compose/gettingstarted/
■ Nginx Beginner's Guide — https://nginx.org/en/docs/beginners_guide.html
■ certbot — Let's Encrypt SSL certificate tool — https://certbot.eff.org/
■ Oracle Cloud Free Tier — always-free 4 OCPU / 24GB VM — https://www.oracle.com/cloud/free/
■ Cloudflare Tunnel — expose local server (free) — https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/
■ ★ Blueprint Chapter 10.1 — full docker-compose.yml with all 7 services — distributed_search_engine_blueprint.pdf
■ ★ Blueprint Chapter 10.2 — Nginx config: gzip, SSL, rate limiting, proxy_pass — distributed_search_engine_blueprint.pdf
■ ★ Blueprint Chapter 10.3 — Free hosting strategy: Oracle Cloud + Cloudflare DNS + Let's Encrypt — distributed_search_engine_blueprint.pdf

✦ Build Tasks:

- ✦ Dockerise crawler, API, and frontend services
- ✦ docker compose up — entire stack starts in one command
- ✦ Nginx: proxy to FastAPI + serve static files + gzip + rate limiting
- ✦ Deploy to Oracle Cloud Free Tier VM — system live at public IP
- ✦ System crawls 100 pages on startup, search returns results in <200ms
- ✦ ★ Use blueprint docker-compose.yml exactly: set replicas: 4 for crawler; verify Kafka receives messages from all replicas
- ✦ ★ Add UptimeRobot free monitoring — ping endpoint every 5 min; keep Railway/Render instance alive
- ✦ ★ Set up Prometheus + Grafana: expose /metrics from FastAPI (prometheus-fastapi-instrumentator); dashboard query P95 latency

■ Week 10 Output: Full system deployed — Nginx + FastAPI + Postgres + Redis + RocksDB in Docker.

POST-WEEK 10 — Evaluation + Advanced Topics

Evaluation Metrics

Implement all offline metrics from blueprint Chapter 8 before running any A/B test.

Metric	Formula	Primary Use
P@k	|relevant ∩ top-k| / k	Precision at rank k
R@k	|relevant ∩ top-k| / |relevant|	Recall at rank k
MAP	Mean of AP over all queries	Ranked retrieval quality
nDCG@10	DCG@10 / IDCG@10	Graded relevance ranking (primary offline metric)
MRR	Mean of 1/rank of first click	First-result quality
CTR@k	Clicks / Impressions at rank k	Online A/B testing
★ Dwell Time	Back-button timestamp - click timestamp	Implicit relevance signal
★ Zero-Result Rate	Queries with 0 clicks / total queries	Index coverage gap detection
★ Query Latency P95	95th percentile response time (ms)	Prometheus + Grafana

★ Blueprint offline_metrics.py implements all of: precision_at_k, recall_at_k, average_precision, mean_average_precision, dcg_at_k, ndcg_at_k, f1_at_k — copy and test each function.

★ Blueprint A/B test: Control (BM25 only) vs Treatment (BM25+PageRank); Mann-Whitney U test after 10K queries/variant; minimum detectable nDCG improvement: 0.5%.

★ Blueprint online metrics collection: JS beacon on click (CTR), unload event for dwell time, ClickHouse for long-click rate aggregation (dwell > 30s).

Advanced Topics Roadmap

Topic	Technology	Description	Resource
Learning-to-Rank	LambdaMART, LambdaRank	Train ranking model from click logs	https://xgboost.readthedocs.io/en/stable/
Vector Search	FAISS / HNSW / Milvus	Dense embeddings for semantic search	https://github.com/facebookresearch/faiss
Neural Ranking	ColBERT, DPR, BGE	BERT-based late interaction models	https://github.com/stanford-futuredata/ColBERT
Distributed Crawling	Consistent Hashing	Partition URL space across machines	https://en.wikipedia.org/wiki/Consistent_hashing
Query Understanding	Intent classif. + NER	Expand, correct, disambiguate queries	https://arxiv.org/abs/2305.04584
★ IndicBERT / AI4Bharat	Multilingual NLP	Indian language semantic search	https://ai4bharat.iitm.ac.in/
★ Spell Correction	SymSpell / n-gram	Edit distance + language model prior	https://github.com/wolfgarbe/SymSpell
★ GPU BM25	CuPy on Oracle A10	GPU-accelerated scoring at scale	https://cupy.dev/
★ Knowledge Graph	Entity linking → Wikidata	Sidebar for recognized entities	https://www.wikidata.org/

Appendix A — Blueprint: System Architecture Reference

Quick-reference tables from the blueprint for use during implementation.

A.1 — System Design Concepts Applied

Concept	Where Used	Tool / Pattern
Load Balancing	Crawler workers, Query replicas	Nginx upstream / Round-robin
Caching	Query results, DNS, robots.txt	Redis (TTL-based)
Message Queue	Crawler → Processor pipeline	Apache Kafka / RabbitMQ
Microservices	Crawler, Indexer, Query, Frontend	Docker Compose / K8s
Sharding	Inverted index partitioning	Hash sharding on term (8 shards)
Consistency	URL dedup, Seen-set	Redis Bloom Filter
Fault Tolerance	Crawler retries, index replication	Retry queues, replica sets
Autoscaling	Crawler workers	Docker Compose scale / K8s HPA
Reverse Proxy	Entry point, SSL termination	Nginx
CDN / Static	Frontend assets	Cloudflare / Nginx static
Rate Limiting	Crawler politeness, API abuse	Token bucket (Redis)

A.2 — Storage Choice Rationale

Data Type	Store	Why
Document metadata	PostgreSQL	Relational, ACID, full-text via tsvector
Inverted posting lists	RocksDB (LSM-tree)	Write-optimized, compressed, SST merges
URL frontier & seen-set	Redis	In-memory speed, BLPOP, Bloom Filter
PageRank scores	PostgreSQL (urls table)	Joins with metadata for reranking
Raw HTML blobs	MinIO (S3-compatible)	Content-addressed, cheap object storage
Session / query cache	Redis	TTL-based result caching
Analytics & A/B logs	ClickHouse	Columnar OLAP for CTR, dwell-time

Appendix B — Blueprint: Evaluation & A/B Testing

B.1 — Online Metrics Collection

Metric	Formula / Definition	Collection Method
Click-Through Rate (CTR)	clicks / impressions per rank position	JS beacon on result click
Dwell Time	time between click and back-button press	JS page unload event + timestamp diff
Mean Reciprocal Rank (MRR)	mean of 1/rank of first click	query_log table
Zero-Result Rate	queries with no clicks / total queries	query_log + click_log JOIN
Query Latency P95	95th percentile response time (ms)	Prometheus + Grafana
Abandonment Rate	queries with no click at all	session analysis in ClickHouse
Long-Click Rate	dwell time > 30s / total clicks	ClickHouse aggregation

B.2 — A/B Testing Setup (Blueprint Chapter 8.4)

Control (A): BM25 only. Treatment (B): BM25 + PageRank reranking. Primary metric: nDCG@10 from implicit feedback (clicks + dwell). Statistical test: Mann-Whitney U (non-parametric). Minimum detectable effect: 0.5% nDCG improvement. Run after 1 week post-deployment with min 10K queries/variant.

Blueprint assign_variant(): stable SHA-256 hash of '{experiment}:{session_id}' — same session always gets same variant. Implement this exactly to avoid session-level variance inflation.

B.3 — 200-Query Benchmark (Indian Languages)

Create a minimum 200-query benchmark with 10 Hindi, 10 Tamil, 10 Telugu, and 10 Bengali queries alongside 160 English queries. Use 3 human judges per query, majority vote for relevance labels (0 = not relevant, 1 = relevant, 2 = highly relevant). Compute nDCG@10 as primary offline metric. See TREC Web Track guidelines for annotation protocols.

■ Deep-Dive Resources:

■ TREC Web Track — official evaluation guidelines — https://trec.nist.gov/data/webmain.html

■ ★ Blueprint Chapter 8.2 — 200-query benchmark procedure — distributed_search_engine_blueprint.pdf

Appendix C — Blueprint: Indian Language NLP Reference

C.1 — Supported Languages

Language	Script	ISO	Tokenizer	Stemmer/Lemma
Hindi	Devanagari	hi	Stanza / IndicNLP	Stanza lemma
Tamil	Tamil	ta	Stanza	Stanza lemma
Telugu	Telugu	te	Stanza	Stanza lemma
Malayalam	Malayalam	ml	Stanza	Stanza lemma
Bengali	Bengali	bn	Stanza	Stanza lemma
Kannada	Kannada	kn	IndicNLP	Rule-based
Gujarati	Gujarati	gu	IndicNLP	Rule-based
Marathi	Devanagari	mr	Stanza	Stanza lemma
Punjabi	Gurmukhi	pa	IndicNLP	Rule-based
Urdu	Nastaliq	ur	IndicNLP	Rule-based

C.2 — Key Libraries

■ Deep-Dive Resources:

■ Stanza — Stanford NLP pipeline for 70+ languages — https://stanfordnlp.github.io/stanza/
■ IndicNLP Library — tokenization + normalization for Indian scripts — https://anoopkunchukuttan.github.io/indic_nlp_library/
■ AI4Bharat IndicBERT — multilingual BERT for Indian languages — https://ai4bharat.iitm.ac.in/
■ indictrans — Indic script transliterator (Roman ↔ Devanagari etc.) — https://github.com/libindic/indic-trans
■ fastText lid.176.bin — language identification model (download) — https://fasttext.cc/docs/en/language-identification.html
■ ★ Blueprint Chapter 7.2 — Unicode NFC normalization + IndicNormalizerFactory — distributed_search_engine_blueprint.pdf
■ ★ Blueprint Chapter 7.3 — Cross-lingual retrieval: Roman Hindi → Devanagari transliteration — distributed_search_engine_blueprint.pdf

C.3 — Cross-Language Retrieval Flow

1. User types 'bharat ki khoj' (Roman-script Hindi). 2. fastText detects language as 'hi' with confidence > 0.7. 3. indictrans.Transliterator converts to Devanagari. 4. Stanza Hindi pipeline tokenizes and lemmatizes. 5. Query fan-out: send to both English shards AND Hindi shards. 6. Score blending: merge result lists, apply language-specific score normalization. 7. Return unified top-10.

■ Deep-Dive Resources:

■ ★ Blueprint Chapter 7.3 — full cross-lingual retrieval flow — distributed_search_engine_blueprint.pdf

---

The Only Rule That Matters

Build it. Ship it. Learn for life. Every concept in this plan has a build task attached. If you finished reading about mutexes without writing a race condition, you haven't learned mutexes. If you finished reading about BM25 without implementing it from scratch, you haven't learned BM25. The search engine you build in 10 weeks will teach you more systems engineering than 2 years of courses alone. Every ★ NEW task is directly traceable to

the blueprint — your north star is a production-grade distributed system, not a tutorial toy. The resources in this plan are the best free materials available on the internet — but they only matter if you close the tab and open a code editor.